LAB 07 — Transformer Architecture

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | **Học kỳ:** Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ **Hạn nộp:** 23:59 - 04/11/2026

Chủ đề: From Attention to Transformer

1. Bối cảnh

Ở LAB 05, sinh viên đã học:

```
Sequence
 ↓
RNN / LSTM / GRU
 ↓
Hidden state
 ↓
Classification
```

LAB 05 cho thấy recurrent models có thể mô hình hóa sequence nhưng phải xử lý thông tin theo từng timestep.

Ở LAB 06, sinh viên chuyển sang attention:

```
Query
 ↓
Key / Value
 ↓
Attention weights
 ↓
Context vector
```

và self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

LAB 07 đặt câu hỏi tiếp theo:

> **Nếu attention có thể cho phép các vị trí trong sequence tương tác trực tiếp với nhau, làm thế nào ghép attention thành một architecture hoàn chỉnh để xử lý văn bản?**

Transformer trả lời bằng cách kết hợp:

```
Input embeddings
      ↓
Positional Encoding
      ↓
Multi-Head Self-Attention
      ↓
Residual + LayerNorm
      ↓
Feed-Forward Network
      ↓
Residual + LayerNorm
      ↓
Transformer Encoder
```

Mục tiêu của LAB 07 không phải là yêu cầu sinh viên tự implement toàn bộ Transformer training system.

Mục tiêu là:

> **Sinh viên phải hiểu từng thành phần cấu thành Transformer và có thể tự ghép một Transformer block tối giản từ các thành phần đã học.**

---

2. Mục tiêu học tập

Sau khi hoàn thành lab, sinh viên có thể:

- **LO1:** Giải thích vì sao Transformer được xây dựng trên self-attention.
- **LO2:** Tính và diễn giải multi-head attention ở mức đơn giản.
- **LO3:** Giải thích tại sao Transformer cần positional information.
- **LO4:** Tính positional encoding cho một số vị trí nhỏ.
- **LO5:** Giải thích vai trò của Feed-Forward Network.
- **LO6:** Giải thích residual connection và LayerNorm.
- **LO7:** Ghép các thành phần thành một Transformer Encoder block.
- **LO8:** Implement một Transformer block tối giản.
- **LO9:** So sánh Transformer với RNN/LSTM.
- **LO10:** Đánh giá ảnh hưởng của positional information.
- **LO11:** Sử dụng Transformer cho text classification.
- **LO12:** Phân tích giới hạn về computational cost và sequence length.
- **LO13:** Chuẩn bị nền tảng cho LAB 08 về pretrained Transformer/BERT.

---

3. Cấu trúc lab

Giai đoạn	Thời lượng	Hoạt động
Theory recap	20 phút	Self-attention và Transformer
Calculation	30 phút	Multi-head và positional encoding
Prediction	10 phút	Transformer behavior
Experiment 1	20 phút	Positional encoding
Core implementation	40 phút	Transformer block
Experiment 2	25 phút	Attention heads
Experiment 3	25 phút	Transformer vs RNN
Application	30 phút	Text classification
Error analysis	20 phút	Model behavior
Reflection	10 phút	Transformer → BERT

---

4. Từ Self-Attention đến Transformer

Self-attention của LAB 06:

$$X \rightarrow Q, K, V \rightarrow Attention \rightarrow Z.$$

Một Transformer block bổ sung các thành phần:

$$Z \rightarrow \text{Residual + Norm} \rightarrow FFN \rightarrow \text{Residual + Norm}.$$

Sơ đồ:

```
Input X
   |
   ▼
Multi-Head Self-Attention
   |
   ▼
Add & Norm
   |
   ▼
Feed-Forward Network
   |
   ▼
Add & Norm
   |
   ▼
Output
```

Đây là **Transformer Encoder block** ở mức khái niệm.

---

5. Tại sao cần Multi-Head Attention?

Một attention head có thể học một kiểu quan hệ.

Ví dụ một head có thể tập trung vào:

```
subject ↔ verb
```

một head khác:

```
negation ↔ sentiment word
```

một head khác:

```
pronoun ↔ entity
```

Không được giả định rằng mỗi head chắc chắn tương ứng với một quy tắc ngôn ngữ cụ thể.

Đây chỉ là motivation trực quan.

Multi-head attention thực hiện nhiều attention projections song song:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V).$$

Sau đó:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O.$$

---

6. Bài tập tính toán — Một attention head

Cho:

$$X \in \mathbb{R}^{3\times 4}.$$

Giả sử một head sử dụng:

$$d_k = 2.$$

Cho:

$$W^Q \in \mathbb{R}^{4\times 2}, \quad W^K \in \mathbb{R}^{4\times 2}, \quad W^V \in \mathbb{R}^{4\times 2}.$$

Sinh viên tính:

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V.$$

Sau đó:

$$S = \frac{QK^T}{\sqrt{2}}$$

và:

$$A = \text{softmax}(S).$$

Cuối cùng:

$$Z = AV.$$

Mục tiêu:

Nhận ra một attention head chỉ là attention của LAB 06 với các projection matrices.

---

7. Bài tập — Multi-head

Giả sử có:

```
2 attention heads
```

và mỗi head tạo ra vector dimension 2.

Khi đó:

$$head_1 \in \mathbb{R}^{T\times 2}$$

và:

$$head_2 \in \mathbb{R}^{T\times 2}.$$

Sau concatenation:

$$Concat(head_1, head_2) \in \mathbb{R}^{T\times 4}.$$

Trả lời:

Vì sao cần concatenation trước khi áp dụng W^O?

---

8. Prediction trước experiment

Sinh viên ghi:

```
Prediction:
Reason:
Confidence:
```

Prediction 1

Một Transformer có thể xử lý các token trong sequence song song hơn RNN không?

Prediction 2

Nếu bỏ positional encoding, Transformer có còn biết thứ tự token không?

Prediction 3

Nếu hai câu có cùng tập token nhưng khác thứ tự, Transformer không có positional information có thể phân biệt chúng không?

Prediction 4

Hai attention heads có nhất thiết học cùng một attention pattern không?

Prediction 5

Tăng số attention heads có chắc chắn làm accuracy tăng không?

Prediction 6

Transformer có luôn nhanh hơn RNN trên mọi sequence length không?

---

9. Positional Encoding

Self-attention tự thân không có khái niệm:

```
first token
second token
third token
```

nếu không cung cấp positional information.

Do đó Transformer cần thêm thông tin vị trí.

Một positional encoding kinh điển:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

và:

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

Input vào Transformer:

$$X' = X + PE.$$

---

10. Bài tập tính positional encoding

Cho:

$$d = 4$$

và:

$$pos = 0, 1, 2.$$

Sinh viên tính positional encoding cho các vị trí tương ứng.

Không yêu cầu tính toàn bộ sequence dài.

Mục tiêu:

Quan sát rằng mỗi vị trí có một vector positional representation khác nhau.

---

11. Experiment 1 — Positional encoding

Tạo một sequence nhỏ:

```
["I", "love", "NLP"]
```

Tạo embedding matrix.

Sau đó:

```
Embedding
    +
Positional Encoding
    ↓
Transformer input
```

So sánh:

```
without positional information
```

và:

```
with positional information
```

Đổi thứ tự:

```
"I love NLP"
```

thành:

```
"NLP love I"
```

Quan sát representation.

Câu hỏi:

> Nếu chỉ dùng self-attention và không có positional information, tại sao thứ tự trở thành vấn đề?

---

12. Permutation experiment

Đây là experiment quan trọng.

Cho:

```
S1 = "dog bites man"
S2 = "man bites dog"
```

Hai sequence chứa cùng vocabulary.

Sinh viên thực hiện:

Experiment A

Self-attention không positional encoding.

Experiment B

Self-attention + positional encoding.

So sánh representation của S1 và S2.

Mục tiêu:

> Demonstrate empirically rằng positional information là thành phần cần thiết để Transformer phân biệt thứ tự.

Không chỉ viết:

> "Transformer cần positional encoding."

Phải có experiment hỗ trợ.

---

13. Feed-Forward Network

Sau self-attention, Transformer áp dụng một position-wise feed-forward network.

Một dạng đơn giản:

$$FFN(x) = W_2\sigma(W_1x + b_1) + b_2.$$

Ví dụ:

```
x
 ↓
Linear
 ↓
ReLU / GELU
 ↓
Linear
 ↓
```

```
output
```

Attention chủ yếu cho phép token tương tác với nhau.

FFN sau đó thực hiện biến đổi phi tuyến trên representation tại từng position.

---

14. Bài tập tính FFN

Cho:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Giả sử:

$$W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

và:

$$b_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Tính:

$$h = ReLU(W_1 x + b_1).$$

Sau đó tính:

$$y = W_2 h + b_2.$$

Mục tiêu:

Hiểu rằng FFN không phải một attention mechanism khác.

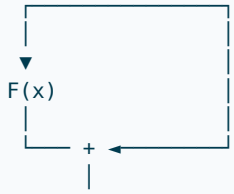
---

15. Residual connection

Transformer sử dụng residual connection:

$$y = x + F(x).$$

Ví dụ:



```
          ▼
       output
```

Trả lời:

Nếu chỉ dùng $F(x)$, điều gì khác với việc sử dụng $x + F(x)$?

Mục tiêu ở lab này là hiểu residual như một đường truyền trực tiếp giúp giữ thông tin đầu vào và hỗ trợ optimization.

---

16. Layer Normalization

Transformer sử dụng normalization bên trong block.

Ở mức trực quan:

```
Input
 ↓
Attention
 ↓
Add & Norm
 ↓
FFN
 ↓
Add & Norm
```

Sinh viên không cần tự implement LayerNorm từ đầu.

Nhưng phải biết LayerNorm chuẩn hóa representation theo feature dimension và được sử dụng để giúp training ổn định.

---

17. Core implementation — Transformer block

Sinh viên xây dựng một Transformer Encoder block tối giản.

Có thể sử dụng các primitive framework:

```
Linear
Softmax
LayerNorm
Dropout
```

nhưng phải tự ghép architecture.

Cấu trúc:

```python
class TransformerBlock:
    def __init__(self, d_model, n_heads, d_ff):
        ...

    def forward(self, x):
        ...
```

Forward:

```
x
 ↓
Multi-Head Self-Attention
 ↓
```

```
Residual + LayerNorm
 ↓
Feed-Forward
 ↓
Residual + LayerNorm
 ↓
output
```

Không được dùng trực tiếp:

```
TransformerEncoderLayer(...)
```

cho phần core implementation.

Có thể sử dụng API đó ở phần application để so sánh.

---

18. Kiểm tra shape

Một phần bắt buộc của implementation là ghi shape.

Giả sử:

$$X \in \mathbb{R}^{B \times T \times d_{model}}.$$

Sinh viên phải ghi:

```
X
↓
Q
↓
K
↓
V
↓
Attention scores
↓
Attention output
↓
FFN
↓
Output
```

Ví dụ:

```
B = 8
T = 100
d_model = 128
n_heads = 4
```

Với:

$$d_k = \frac{128}{4} = 32.$$

Phải xác định được:

$$Q, K, V \in \mathbb{R}^{8 \times 4 \times 100 \times 32}.$$

Attention scores:

$$\mathbb{R}^{8\times4\times100\times100}.$$

Đây là một phần rất quan trọng vì shape errors là một trong những lỗi phổ biến khi tự implement Transformer.

---

19. Experiment 2 — Number of attention heads

Giữ:

$$d_{model} = 128.$$

Thử:

```
n_heads = 1
n_heads = 2
n_heads = 4
n_heads = 8
```

Do đó:

$$d_k = \frac{d_{model}}{n_{heads}}.$$

Báo cáo:

Heads	d_k	Parameters	Training time	Accuracy	F1
1	128				
2	64				
4	32				
8	16				

Câu hỏi:

Tăng số heads có đồng nghĩa với tăng representation dimension không?

Và:

Khi d_{model} cố định, tăng số heads làm d_k thay đổi như thế nào?

---

20. Experiment 3 — Attention head diversity

Với model có nhiều heads, visualize attention matrix của từng head.

Ví dụ:

```
Head 1
Head 2
Head 3
Head 4
```

Không yêu cầu đặt tên:

```
Head 1 = syntax
Head 2 = semantics
```

trừ khi có evidence cụ thể.

Sinh viên chỉ cần mô tả:

- head nào tập trung mạnh;
- head nào phân tán;
- các heads có giống nhau không;
- có head nào gần như redundant không.

Câu hỏi:

> Multi-head attention có thực sự tạo ra các attention patterns khác nhau trong experiment của bạn không?

---

21. Experiment 4 — Transformer vs RNN/LSTM

Sử dụng cùng dataset classification của LAB 05.

So sánh:

```
Model A
Embedding
 ↓
LSTM
 ↓
Classifier
```

với:

```
Model B
Embedding
 ↓
Positional Encoding
 ↓
Transformer Encoder
 ↓
Pooling
 ↓
Classifier
```

Giữ cố định càng nhiều yếu tố càng tốt:

- dataset;
- train/validation/test split;
- vocabulary;
- embedding dimension;
- batch size;
- optimizer;
- learning rate;
- number of epochs.

Báo cáo:

Model	Parameters	Training time	Accuracy	Precision	Recall	F1
LSTM						
Transformer						

Không đặt mục tiêu rằng Transformer phải thắng.

Câu hỏi:

> Sự khác biệt về performance có thể xuất phát từ những thành phần architecture nào?

---

22. Experiment 5 — Sequence length

Thử:

```
max_length = 32
max_length = 64
max_length = 128
max_length = 256
```

Báo cáo:

Length	Tokens retained	Attention memory	Training time	F1
32				
64				
128				
256				

Đặc biệt quan sát:

$$T^2.$$

Nếu sequence tăng:

$$128 \to 256,$$

số phần tử trong attention matrix tăng:

$$128^2 \to 256^2.$$

Tức là:

$$16,384 \to 65,536.$$

Trả lời:

> Tại sao Transformer không đơn giản giải quyết mọi vấn đề bằng cách tăng sequence length?

---

23. Application — Transformer text classification

Xây dựng:

```
Raw text
 ↓
Tokenizer
 ↓
Token IDs
```

```
 ↓
Embedding
 ↓
Positional Encoding
 ↓
Transformer Encoder
 ↓
Pooling
 ↓
Linear classifier
 ↓
Prediction
```

Dataset:

IMDb

Task:

```
positive
negative
```

Có thể tiếp tục dùng dataset LAB 05 để kiểm soát biến thay đổi.

Input:

```
"I really enjoyed this movie."
```

Output:

```
Positive
Probability: ...
```

Sinh viên phải test ít nhất:

- câu ngắn;
- câu dài;
- negation;
- contrast;
- ambiguous sentiment.

---

24. Pooling

Transformer Encoder tạo representation cho từng token:

$$H \in \mathbb{R}^{T \times d}.$$

Để classification cần biến thành representation của toàn sequence.

Một cách đơn giản:

$$h_{mean} = \frac{1}{T} \sum_{t=1}^{T} h_t.$$

Có thể sử dụng:

```
mean pooling
```

hoặc một token đặc biệt:

```
[CLS]
```

nếu architecture/tokenizer được thiết kế cho nó.

Ở LAB 07, sinh viên nên thử **mean pooling trước** để tránh đưa quá nhiều khái niệm mới cùng lúc.

---

25. Experiment 6 — Pooling strategy

So sánh:

```
Mean pooling
```

và:

```
First-token / CLS-style pooling
```

nếu model được thiết kế phù hợp.

Báo cáo:

Pooling	Accuracy	F1	Training time
Mean			
CLS-style			

Câu hỏi:

> Cách lấy sequence representation có ảnh hưởng tới classification không?

---

26. Error analysis

Chọn:

```
3 correct
3 incorrect
```

Phân tích:

```
negation
contrast
long sequence
truncation
rare word
ambiguous sentiment
domain-specific term
noise
```

Đối với Transformer, bổ sung:

> Các token quan trọng có xuất hiện trong attention pattern hay không?

Nhưng phải tránh kết luận:

> "Attention = explanation."

Sinh viên cần phân biệt:

```
model behavior
```

với:

```
human interpretation
```

Attention visualization chỉ cho thấy phân phối attention weights.

---

27. So sánh toàn bộ LAB 04 → LAB 07

Property	TF-IDF	Avg Word2Vec	LSTM	Attention	Transformer
Sparse	v				
Dense		v	v	v	v
Word order	Limited	x	v	v	v*
Context-dependent	x	x	v	v	v
Direct token interaction	x	x	Limited	v	v
Sequential computation			v		
Parallelizable	v	v	Limited	v	v
Long-range interaction	Limited	x	Difficult	Improved	Improved
Attention	x	x	x	v	v
Positional information			Implicit	Required	Required
Cost with sequence length			~linear	$O(T^2)$	$O(T^2)$

∗ Transformer cần positional information để biểu diễn thứ tự.

Câu hỏi tổng hợp:

Từ LAB 01 tới LAB 07, mỗi bước đã giải quyết limitation nào của bước trước?

Sinh viên phải hoàn thành:

```
TF-IDF
   ↓
Limitation:
__________

Word2Vec
   ↓
Limitation:
__________

LSTM
   ↓
Limitation:
__________

Attention
   ↓
Limitation:
__________

Transformer
   ↓
Remaining limitation:
```

```
_________
```

---

28. Prediction verification

Quay lại sáu prediction đầu lab.

Bảng:

Prediction	Dự đoán	Kết quả	Đúng/Sai	Giải thích
1				
2				
3				
4				
5				
6				

Sinh viên bắt buộc phải giải thích prediction sai.

Không được sửa prediction ban đầu sau khi xem kết quả.

---

29. Learning curves

Báo cáo:

```
training loss
validation loss
```

theo epoch.

Kiểm tra:

- overfitting;
- underfitting;
- convergence;
- instability.

Ví dụ:

```
Train loss       ↓↓↓↓↓

Validation loss  ↓↓↓ → ↑↑
```

có thể cho thấy validation performance bắt đầu xấu đi trong khi training tiếp tục cải thiện.

---

30. Individual learning check

Giảng viên random một câu trong 3 phút.

Ví dụ:

Tại sao Transformer cần positional encoding?

hoặc:

Multi-head attention khác một attention head ở đâu?

hoặc:

Nếu $d_{model} = 128$ và có 8 heads thì d_k bằng bao nhiêu?

hoặc:

Tại sao attention matrix có kích thước $T \times T$?

hoặc:

Residual connection có vai trò gì?

hoặc:

Feed-Forward Network làm gì trong Transformer block?

hoặc:

Tại sao Transformer có thể xử lý các token song song hơn RNN?

hoặc:

> Nếu sequence dài gấp đôi, attention matrix thay đổi như thế nào?

hoặc:

> Vì sao không thể kết luận rằng attention weight cao đồng nghĩa token đó là "lý do" model đưa ra prediction?

---

31. Deliverables

```
lab07/
├── README.md
├── calculations.md
├── prediction.md
├── transformer_block.py
├── positional_encoding.py
├── transformer_classification.ipynb
├── results.csv
├── positional_encoding.png
├── attention_heads.png
├── learning_curves.png
├── confusion_matrix.png
├── error_analysis.md
└── reflection.md
```

---

32. Rubric

Thành phần	Điểm
Theory & calculation	15
Prediction	10
Positional encoding experiment	10
Transformer block implementation	20
Multi-head experiment	10
RNN/LSTM vs Transformer comparison	10
Sequence-length experiment	10
Application & evaluation	5
Error analysis	5
Reflection	5
Tổng	**100**

---

33. Peer assessment

Reviewer kiểm tra:

```
Theory
[ ] Giải thích được self-attention
[ ] Giải thích được multi-head attention
[ ] Giải thích được positional encoding
[ ] Giải thích được FFN
[ ] Giải thích được residual + LayerNorm
```

```
Calculation
[ ] Q/K/V shapes đúng
[ ] Multi-head shapes đúng
[ ] Positional encoding đúng

Implementation
[ ] Transformer block hoạt động
[ ] Residual connection đúng
[ ] LayerNorm đúng vị trí
[ ] FFN đúng câú trúc

Experiment
[ ] Có positional encoding experiment
[ ] Có attention-head experiment
[ ] Có sequence-length experiment
[ ] Có LSTM/Transformer comparison

Analysis
[ ] Learning curves
[ ] Confusion matrix
[ ] Ít nhất 3 lỗĩ
[ ] Không đồng nhất attention với explanation

Understanding
[ ] Giải thích được $T\times T$
[ ] Giải thích được $d_k$
[ ] Giải thích được multi-head
[ ] Giải thích được positional information
```

---

34. Mạch nối LAB 01 → LAB 07

Đến đây curriculum có một progression rõ ràng:

```
LAB 01
Text
 ↓
TF-IDF
 ↓
Sparse representation
 ↓
Search
```

```
LAB 02
Text
 ↓
N-gram
 ↓
Probability
 ↓
Language Model
 ↓
Perplexity
```

```
LAB 03
Text
 ↓
Context
 ↓
```

```
Word Embedding
 ↓
Semantic Similarity
```

```
LAB 04
Text
 ↓
Representation
 ↓
Classifier
 ↓
Evaluation
 ↓
Error Analysis
```

```
LAB 05
Sequence
 ↓
RNN / LSTM / GRU
 ↓
Hidden state
 ↓
Sequence classification
```

```
LAB 06
Sequence
 ↓
Query / Key / Value
 ↓
Attention
 ↓
Context representation
```

```
LAB 07
Sequence
 ↓
Self-Attention
 ↓
Multi-Head Attention
 ↓
Positional Encoding
 ↓
FFN
 ↓
Residual + LayerNorm
 ↓
Transformer
```

LAB 07 tạo ra một thay đổi quan trọng trong cách sinh viên nhìn NLP:

$$\boxed{\text{Sequence modeling} \rightarrow \text{Direct token interaction}}$$

và:

$$\boxed{\text{Attention} + \text{Position} + \text{FFN} + \text{Residual/Norm} \rightarrow \text{Transformer}}$$

Nhưng Transformer ở LAB 07 vẫn là **architecture học từ đầu**.

Sinh viên chưa trả lời được một câu hỏi rất quan trọng:

> Nếu Transformer có thể học representation mạnh như vậy, tại sao không dùng một Transformer đã được huấn luyện trên một lượng văn bản khổng lồ rồi fine-tune cho task cụ thể?

Đó là động lực tự nhiên để LAB 08 chuyển sang:

```
LAB 08

Pretrained Transformer
        ↓
BERT-style Encoder
        ↓
Pretraining
        ↓
Fine-tuning
        ↓
Text Classification / NER
```

Điểm này cũng giúp tránh một lỗi thiết kế curriculum thường gặp: **dạy BERT trước khi sinh viên hiểu Transformer block**. Sau LAB 07, khi sang BERT, sinh viên đã biết `[CLS]`, positional information, self-attention, FFN, residual và LayerNorm đang nằm ở đâu trong architecture.